HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
VIỆN KHOA HỌC KỸ THUẬT BƯU ĐIỆN

BÁO CÁO NỘI DUNG

Nội dung 4: Xây dựng AI Agent tóm tắt biên bản và module đồng bộ

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN NĂM 2026

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG MEET-ASSISTANT: TỰ ĐỘNG HÓA TAKE-NOTE VÀ ĐỒNG BỘ HÓA QUẢN LÝ CÔNG VIỆC

18-SV-2026-RIPT

Thuộc nhóm ngành khoa học: Công nghệ thông tin

Chủ trì đề tài: Trần Đức Định (B23DCCC038)
Thành viên: Hoàng Nguyễn Trà My (B23DCCC116)
Giảng viên hướng dẫn: TS. Lê Văn Hoàng

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG.....1
1.1. Nhiệm vụ theo đề cương.....1
1.2. Phương pháp thực hiện.....1
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....2
2.1. Thiết kế bộ phân tích AI.....2
2.1.1. Quy tắc chống bịa trong prompt.....2
2.1.2. Cấu trúc đầu ra.....2
2.1.3. Con người trong vòng lặp.....3
2.2. Thiết kế module đồng bộ công cụ ngoài và cảnh báo.....3
2.2.1. Hiện trạng và dữ liệu.....3
2.2.2. Nguyên tắc chống tạo trùng và cảnh báo.....4
CHƯƠNG 3. XÂY DỰNG HỆ THỐNG.....5
3.1. Mô-đun phân tích biên bản bằng LLM.....5
3.2. Luồng duyệt và quản lý công việc.....6
3.3. Mô-đun đồng bộ Trello và cảnh báo Telegram.....7
3.4. Giao diện web.....10
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ.....15
4.1. Lựa chọn mô hình ngôn ngữ lớn.....15
4.2. Đánh giá đầu-cuối từ âm thanh đến công việc.....16
CHƯƠNG 5. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ.....19
5.1. Đối chiếu với kết quả cần đạt.....19
5.2. Tồn tại và hướng hoàn thiện.....19
TÀI LIỆU THAM KHẢO.....21

DANH MỤC BẢNG BIỂU

Bảng 2.1. Cấu trúc JSON đầu ra của bộ phân tích.....2
Bảng 2.2. Các bảng của schema integration.....3
Bảng 3.1. Hành vi của mô-đun đồng bộ theo từng nhà cung cấp.....8
Bảng 4.1. So sánh các LLM trên bài trích xuất biên bản (transcript chuẩn, tối đa 15 điểm) 15
Bảng 4.2. Điểm trích xuất khi đầu vào là transcript do máy nhận dạng (tối đa 15).....16
Bảng 5.1. Đối chiếu kết quả Nội dung 4 với đề cương.....19

DANH MỤC HÌNH VẼ

Hình 3.1. Trích xuất biên bản có cấu trúc bằng LLM và đối chiếu bằng chứng.....5
Hình 3.2. Ngăn chi tiết công việc hiển thị kết quả đồng bộ Trello và Telegram.....8
Hình 3.3. Màn hình tổng quan của người quản lý dự án.....11
Hình 3.4. Danh sách cuộc họp với bộ lọc và giai đoạn xử lý.....11
Hình 3.5. Màn hình transcript với nhãn ngôn ngữ, cảnh báo nói chồng và gán người nói.12
Hình 3.6. Màn hình biên bản và vòng duyệt theo phiên bản.....12
Hình 3.7. Hàng đợi duyệt đề xuất công việc do AI trích xuất.....13
Hình 3.8. Bảng công việc theo trạng thái.....13
Hình 3.9. Hộp thoại mời bot ghi âm vào cuộc họp Google Meet.....14
Hình 4.1. Độ đúng trích xuất và độ trễ của các LLM trên FPT AI Factory.....15

CHƯƠNG 1. GIỚI THIỆU NỘI DUNG

1.1. Nhiệm vụ theo đề cương

Nội dung 4 được thực hiện trong tháng 09/2026. Theo đề cương, nhiệm vụ của bước này là xây dựng AI Agent tóm tắt và đồng bộ: triển khai LLM để trích xuất các đầu việc (action items), xây dựng module kết nối và tự động đẩy công việc sang các công cụ quản lý dự án. Kết quả cần đạt gồm Agent tự động tạo biên bản họp, đồng bộ thành công công việc lên Jira/Trello và hệ thống cảnh báo công việc mới qua Telegram.

Điểm mấu chốt về thiết kế của nội dung này là AI chỉ đề xuất, con người quyết định. Biên bản và công việc do LLM sinh ra có thể sai (bịa người thực hiện, bịa thời hạn, nhầm đề xuất thành quyết định), nên kết quả của AI phải đi qua hàng đợi duyệt; chỉ công việc đã được trưởng nhóm duyệt mới trở thành công việc chính thức và mới được phép đồng bộ ra công cụ bên ngoài.

1.2. Phương pháp thực hiện

Bộ phân tích được thiết kế sau một giao diện chung để luồng nghiệp vụ không phụ thuộc nhà cung cấp LLM. Trong tháng 09/2026, nhóm hoàn thành toàn bộ luồng từ transcript đến biên bản nháp, hàng đợi duyệt và công việc chính thức với một bộ phân tích giả lập có đầu ra cố định, được chọn để kiểm tra các quy tắc nghiệp vụ. LLM thật (qua dịch vụ FPT AI Factory) được gắn vào sau, và mô hình được chọn bằng thực nghiệm so sánh có đáp án. Phần đồng bộ được thiết kế dữ liệu và luồng sự kiện trước, sau đó hiện thực trong integration-service ở giai đoạn hoàn thiện (đầu tháng 11/2026) cho Trello và Telegram, theo đúng hai nguyên tắc đã đặt: chỉ đồng bộ công việc đã duyệt và thử lại không tạo bản trùng. Mô-đun được kiểm thử bằng kiểm thử đơn vị với các lời gọi API giả lập, rồi chạy đầu-cuối với tài khoản Trello và bot Telegram thật; đồng bộ Jira giữ ở mức thiết kế.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Thiết kế bộ phân tích AI

2.1.1. Quy tắc chống bịa trong prompt

Bộ phân tích dùng mô hình ngôn ngữ lớn [3] qua FPT AI Factory [5] với giao diện tương thích OpenAI; mô hình chính là DeepSeek-V4.1-Flash [4], dự phòng là GLM-5.2 [8]. Mô hình chính lỗi thì chuyển sang dự phòng; cả hai lỗi thì phát `analysis.failed`. Transcript đưa vào dạng các dòng đánh số `[số thứ tự] Người nói: lời nói`, để mô hình chỉ ra bằng chứng bằng một con số kiểm tra được. Lời nhắc hệ thống đặt mô hình vào vai thư ký và nêu các quy tắc chống bịa [7]:

- Quyết định chỉ gồm điều cả nhóm đã chốt; đề xuất chưa ai đồng ý ("hay là…") không phải quyết định.
- Người thực hiện ghi đúng tên trong transcript; không rõ thì để trống.
- Hạn chót ghi nguyên văn ("thứ Sáu", "18/11"); không có thì để trống, không tự đoán ngày.
- Có đính chính (đổi hạn, đổi người, huỷ việc) thì lấy thông tin sau cùng; bỏ qua chuyện ngoài lề.
- Mỗi mục kèm câu trích nguyên văn trong đúng dòng được nêu; không có gì thì trả mảng rỗng.

2.1.2. Cấu trúc đầu ra

Mô hình chỉ trả một đối tượng JSON như Bảng 2.1. Chúng tôi không bật chế độ ép JSON của API, vì khi bật, DeepSeek-V4.1-Flash trả nội dung rỗng; bộ phân tích tự bóc đối tượng JSON từ câu trả lời và tự kiểm tra cấu trúc.

Bảng 2.1. *Cấu trúc JSON đầu ra của bộ phân tích*

Trường	Ý nghĩa
`summary`	Tóm tắt 3–5 câu
`decisions[]`	title, description, quote, transcriptSequence
`openIssues[]`	Vấn đề chưa có kết luận
`actionItems[].assigneeHint`	Tên người thực hiện như trong transcript, hoặc rỗng
`actionItems[].deadlineRawText`	Cụm chỉ thời hạn nguyên văn, hoặc rỗng
`actionItems[].priority`, `confidence`	high/medium/low; số trong [0, 1]
`actionItems[].quote`, `transcriptSequence`	Câu trích làm bằng chứng và số thứ tự dòng

Kết quả sau đó được chuẩn hoá: ưu tiên không hợp lệ đưa về `medium`; độ tin cậy ép vào [0, 1]; các chuỗi "null", "N/A" đưa về rỗng thật; mục thiếu tiêu đề bị loại. Quan trọng nhất là bước *đối chiếu bằng chứng*: mô hình đôi khi trích đúng câu nhưng ghi lệch số dòng, nên bộ phân tích so câu trích với dòng được nêu, không khớp thì dò lại toàn transcript, vẫn không thấy thì chỉ giữ số dòng nếu số đó tồn tại. Transcript rỗng thì không gọi mô hình.

2.1.3. Con người trong vòng lặp

Đầu ra không bao giờ thành dữ liệu chính thức ngay: Core chuyển nó thành biên bản nháp và các đề xuất giữ bản gốc; người duyệt xem đề xuất cạnh câu nguồn, chọn người thực hiện từ danh sách thành viên (`assigneeHint` chỉ để tham khảo), chọn ngày hạn rồi duyệt hoặc từ chối. Mô hình được coi như người soạn nháp nhanh nhưng có thể sai, vì một công việc giao nhầm người hay nhầm hạn gây hậu quả lớn hơn nhiều so với vài giây kiểm tra.

2.2. Thiết kế module đồng bộ công cụ ngoài và cảnh báo

2.2.1. Hiện trạng và dữ liệu

Module đồng bộ công việc sang Jira [1] hoặc Trello [2] và gửi cảnh báo qua Telegram [9] thuộc integration-service. Đến thời điểm nghiệm thu, phần Trello và Telegram đã được hiện thực ở mức MVP (trình bày ở chương Xây dựng hệ thống): integration-service nhận `task.sync.requested`, tạo thẻ Trello, gửi tin Telegram rồi phát `task.sync.completed`; bốn bảng của schema `integration` ở Bảng 2.2 đã được tạo. Phần còn ở mức thiết kế gồm đồng bộ Jira, màn hình kết nối theo từng workspace (trang "Tích hợp" vẫn là giao diện mẫu) và đồng bộ hai chiều trạng thái. Mọi trường trỏ về Core đều là tham chiếu logic, không có khoá ngoại xuyên schema.

Bảng 2.2. *Các bảng của schema integration*

Bảng	Cột quan trọng	Vai trò
`integrations`	provider, status, config, lastError	Một kết nối với một nhà cung cấp; `config` chỉ giữ thông tin không bí mật (danh sách Trello đã dò, cuộc trò chuyện Telegram)
`external_task_links`	integrationId, actionItemId, externalId, externalUrl, syncStatus	Liên kết công việc với thẻ ngoài; duy nhất (integrationId, actionItemId)
`sync_jobs`	type, status, idempotencyKey, payload, result, retryCount, error	Một lần đẩy công việc ra một nhà cung cấp; duy nhất idempotencyKey
`processed_messages`	messageId, consumer	Chống xử lý lặp thông điệp RabbitMQ; duy nhất (messageId, consumer)

Luồng thiết kế: khi công việc được duyệt, Core ghi `task.sync.requested` vào outbox; integration-service tạo job, gọi REST API của nền tảng đích, ghi liên kết ngoài rồi phát `task.sync.completed`. Khi hiện thực, nhóm điều chỉnh một điểm so với thiết kế ban đầu: thay vì để integration-service gọi ngược API nội bộ của Core để đọc công việc, thông điệp mang sẵn ảnh chụp công việc, tránh phụ thuộc đồng bộ giữa hai dịch vụ. Theo thiết kế đầy đủ, kết nối dùng OAuth 2.0 [6] nơi nhà cung cấp hỗ trợ, token được mã hoá khi lưu và không bao giờ được trả về qua API hay giao diện; bản MVP hiện tại lấy khoá API từ biến môi trường và không ghi khoá vào cơ sở dữ liệu.

2.2.2. Nguyên tắc chống tạo trùng và cảnh báo

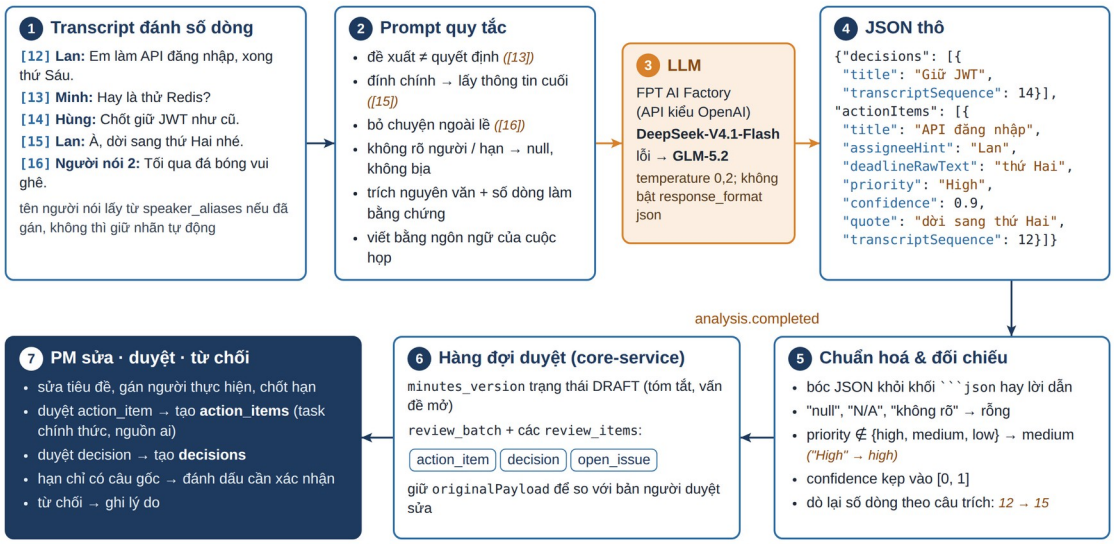
Hai nguyên tắc chi phối đồng bộ. Thứ nhất, *chỉ đồng bộ công việc đã duyệt*, vì một thẻ hay issue sai trên công cụ ngoài khó thu hồi hơn một đề xuất bị từ chối. Thứ hai, *thử lại không tạo bản trùng*: chống lặp thông điệp nội bộ là chưa đủ, vì lời gọi tạo issue có thể đã thành công nhưng phản hồi bị mất, gửi lại sẽ tạo issue thứ hai. Với API không hỗ trợ khoá idempotency, thiết kế yêu cầu tra cứu đối soát trước khi thử lại; ràng buộc duy nhất trên (kết nối, công việc) là chốt chặn cuối. Bản hiện thực cho Trello áp dụng đúng nguyên tắc này bằng một dòng đánh dấu trong mô tả thẻ; phạm vi đã hiện thực là đồng bộ một chiều (chỉ tạo thẻ).

Cảnh báo dựa trên quy tắc minh bạch thay vì phán đoán của AI. Bảng `risk_flags` đã có trong schema `core`, lưu quy tắc sinh cảnh báo (`ruleKey`), dữ liệu căn cứ (`evidence`) và khoá chống tạo trùng qua các lần quét. Hiện giao diện tính năm loại cảnh báo trên dữ liệu công việc (quá hạn, bị chặn, thiếu người phụ trách, phụ thuộc chưa xong, từ năm hạn dồn vào một ngày cho một người), mỗi cảnh báo kèm quy tắc tương ứng; chỉ kết luận "quá tải" khi có dữ liệu công sức và năng lực. Kênh Telegram hiện mới dùng để báo công việc mới được duyệt; bộ quét phía máy chủ và các cảnh báo, nhắc việc theo quy tắc (quá hạn, sắp đến hạn) qua Telegram chưa được hiện thực.

CHƯƠNG 3. XÂY DỰNG HỆ THỐNG

3.1. Mô-đun phân tích biên bản bằng LLM

Mô-đun phân tích nhận transcript và trả về tóm tắt, quyết định, vấn đề còn mở và công việc, mỗi mục kèm câu trích và số dòng làm bằng chứng (Hình 3.1). Worker gọi dịch vụ Serverless Inference của FPT AI Factory [5] qua API kiểu OpenAI tại `https://token-api.fpt.ai/v1`, nhiệt độ 0,2, tối đa 32.000 token đầu ra vì mô hình có bước suy luận tiêu tốn nhiều token. Nhóm không bật `response_format` JSON vì khi bật, mô hình chính trả nội dung rỗng; prompt yêu cầu JSON và worker tự bóc tách, kể cả khi mô hình bọc kết quả trong khối mã hay nói thêm.



Hình 3.1. *Trích xuất biên bản có cấu trúc bằng LLM và đối chiếu bằng chứng*

Transcript được đưa vào dạng các dòng `[số thứ tự] Người nói: lời nói`. Lời nhắc hệ thống, dựa trên khả năng làm theo chỉ dẫn của LLM [3], quy định viết biên bản bằng ngôn ngữ chính của cuộc họp và nêu các quy tắc rút từ tình huống thật: đề xuất chưa ai đồng ý ("hay là…") không phải quyết định; người thực hiện ghi đúng tên trong transcript; thời hạn giữ nguyên văn ("thứ Sáu"), không tự đoán ngày; có đính chính thì lấy thông tin cuối; bỏ chuyện ngoài lề; không bịa thông tin [7]; câu trích phải nguyên văn từ đúng dòng được dẫn.

Mô hình được chọn bằng chấm thử có đáp án trên một cuộc họp tiếng Việt mẫu chứa đủ hiện tượng khó (đề xuất khác quyết định, hạn bị đính chính, đổi người làm giữa chừng, chuyện ngoài lề, câu tiếng Anh xen kẽ, người nói chỉ có nhãn tự động); mỗi mô hình ứng viên chạy nhiều lần, chấm theo đáp án và đo thời gian. Chỉ xét mô hình do FPT tự vận hành để dữ liệu hội thoại không ra ngoài hạ tầng FPT. Kết quả, DeepSeek-V4.1-Flash (họ DeepSeek [4]) được chọn làm mô hình chính vì chính xác

nhất dù chậm hơn, GLM-5.2 (họ GLM [8]) nhanh hơn nhiều làm dự phòng; số liệu ở chương Thực nghiệm và đánh giá (Hình 4.1).

Lời gọi LLM dùng chế độ truyền luồng (stream): khi gọi thường, cổng dịch vụ trả lỗi hết thời gian chờ (HTTP 524) nếu mô hình suy luận quá lâu, còn ở chế độ luồng các token suy luận chảy về liên tục nên kết nối không bị cắt; phần suy luận bị bỏ qua, chỉ ghép phần nội dung, và tổng thời gian vẫn bị giới hạn. Worker thử mô hình chính rồi mô hình dự phòng; một mô hình bị coi là không dùng được khi lỗi HTTP, trả nội dung rỗng, hết token trước khi xong JSON hoặc JSON hỏng; cả hai lỗi thì phát `analysis.failed` kèm lỗi của cả hai. Kết quả được chuẩn hoá: bỏ mục không tiêu đề, quy "null", "N/A" về rỗng, `priority` ngoài {high, medium, low} quy về medium, `confidence` cắt vào [0, 1]. Số dòng bằng chứng được đối chiếu lại với câu trích, vì mô hình đôi khi trích đúng câu nhưng ghi lệch dòng:

```
def ground(quote, sequence, texts):
    needle = _norm(quote).strip(" .,\"'“”…") if quote else ""
    if needle:
        if sequence in texts and needle in _norm(texts[sequence]):
            return sequence                # số dòng model đưa là đúng
        for seq, text in texts.items():    # dò lại theo nội dung câu trích
            if needle in _norm(text):
                return seq
    return sequence if sequence in texts else None
```

Khi transcript rỗng, worker không gọi LLM mà trả kết quả rỗng, tránh để mô hình bịa biên bản. Cấu hình đọc từ biến môi trường; khoá API đặt trong tệp `.env` đã đưa vào `.gitignore`, thiếu khoá thì worker tự lùi về bộ phân tích giả lập. Tên người thực hiện (`assigneeHint`) chỉ là tên trong transcript; Core không tự ánh xạ sang tài khoản mà để người duyệt chọn.

3.2. Luồng duyệt và quản lý công việc

Kết quả AI không trở thành dữ liệu chính thức nếu chưa có người duyệt. Biên bản quản lý theo phiên bản với ba trạng thái `draft` → `pending_approval` → `approved`; người có quyền `minutes.approve` duyệt hoặc trả về nháp. Biên bản đã duyệt không sửa trực tiếp: thao tác "tạo nháp mới" sinh phiên bản kế tiếp chép nội dung bản đã duyệt, bản cũ giữ nguyên; chạy lại AI cũng chỉ tạo nháp mới.

Đề xuất của AI nằm trong `review_items` với ba loại: công việc, quyết định, vấn đề còn mở; mỗi mục giữ song song bản AI sinh và bản người dùng sửa. Duyệt một đề xuất công việc sinh đúng một `action_items` nguồn `ai`, ghi người duyệt, độ tin cậy, mức ưu tiên, người thực hiện do người duyệt chọn; nếu chỉ có câu gốc như "thứ Sáu" mà chưa chốt ngày thì task được đánh dấu cần xác nhận hạn. Duyệt đề xuất quyết định sinh một `decisions`. Nguồn gốc của task được truy qua đề xuất đã sinh ra nó: câu

trích và số dòng nằm trong nội dung đề xuất, lô duyệt giữ định danh phiên bản transcript nguồn; bảng `action_item_citations` đã có nhưng thao tác duyệt chưa ghi vào — phần này đang hoàn thiện. Sau khi duyệt, PM/Admin hoặc người được giao đổi được trạng thái task, còn giao người, đổi hạn, đổi ưu tiên chỉ người có quyền `task.assign` làm được; dịch vụ chỉ coi là "lên kế hoạch" khi giá trị thật sự khác giá trị đang lưu, vì giao diện gửi lại cả trường không đổi.

Gán tên người nói (speaker alias) là thao tác có chủ đích của người dùng. Khi gắn nhãn "Người nói 2" với một thành viên, các đoạn transcript chuyển sang danh tính `confirmed` và `speaker_aliases` ghi người xác nhận. Thao tác này **không** tự đổi người được giao của task đã duyệt; thay vào đó, task của cuộc họp được đánh dấu `needsReviewReason = speaker_changed` để PM rà soát.

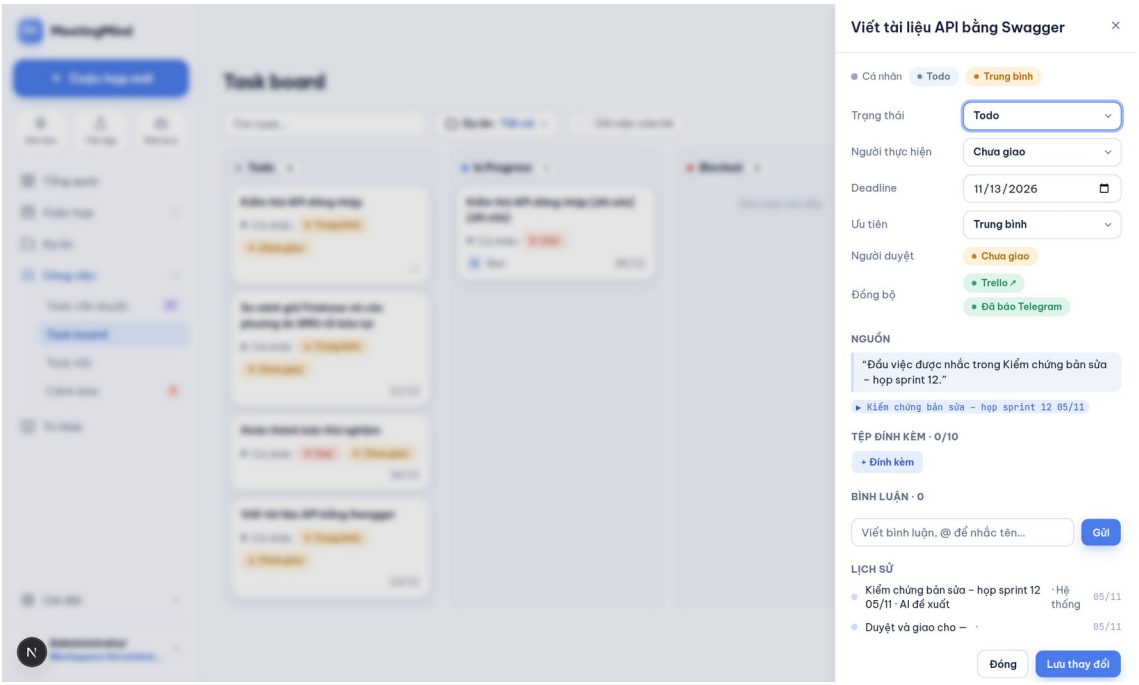
Vì phân tích AI chạy ngay khi có transcript, tức là trước khi người dùng kịp gán tên, hệ thống cung cấp thao tác **phân tích lại** (`POST /workflow/meetings/:id/reanalyze`, nút "Phân tích lại" trong khung người nói). Khi gán tên, bí danh lưu tên hiển thị của thành viên; khi phân tích lại, transcript gửi sang AI mang tên thật thay cho nhãn "Người nói N", nhờ đó người thực hiện trong đề xuất mới là tên thành viên. Mỗi lần phân tích lại là một job mới với khoá chống lặp riêng, sinh biên bản nháp và lô đề xuất mới; biên bản, công việc đã duyệt giữ nguyên.

3.3. Mô-đun đồng bộ Trello và cảnh báo Telegram

Mô-đun này hiện thực phần đồng bộ trong thiết kế ở chương Phân tích và thiết kế: đưa công việc *đã được duyệt* sang bảng Trello [2] của nhóm và báo tin qua Telegram [9]. Mô-đun nằm trong integration-service — dịch vụ NestJS trước đó chỉ là khung — dưới tên `task-sync`, và được làm ở mức MVP: mỗi nhà cung cấp một kết nối cấu hình qua biến môi trường, đồng bộ một chiều (chỉ tạo thẻ, chưa cập nhật trạng thái ngược lại); đồng bộ Jira chưa được hiện thực.

Luồng xử lý. Khi người duyệt chấp nhận một đề xuất công việc, cùng với việc tạo `action_items`, Core ghi sự kiện `task.sync.requested` vào outbox. Vì integration-service không được đọc schema `core`, thông điệp mang ảnh chụp công việc tại lúc duyệt: tiêu đề, mô tả, tên người thực hiện, hạn dạng ISO 8601 và câu hạn gốc, mức ưu tiên, tên cuộc họp, câu trích nguồn, cùng khoá `<actionItemId>:push`. Bộ phát outbox đẩy sự kiện lên exchange topic `meetingmind`; integration-service, nay đã bật kết nối RabbitMQ, nhận nó qua hàng đợi bền `integration.jobs.task-sync` rồi xử lý lần lượt Trello và Telegram. Sau khi ghi kết quả vào cơ sở dữ liệu, dịch vụ phát `task.sync.completed` gồm `actionItemId` và kết quả của từng nhà cung cấp (trạng thái `synced`, `failed` hoặc `skipped`, mã và đường dẫn đối tượng ngoài, thông

báo lỗi). Core lưu danh sách này vào trường `externalLinks` của công việc; ngăn chi tiết công việc trên giao diện có thêm dòng "Đồng bộ" với liên kết "Trello" dẫn tới thẻ, nhãn "Đã báo Telegram" hoặc lỗi của từng bên (Hình 3.2).



Hình 3.2. *Ngăn chi tiết công việc hiển thị kết quả đồng bộ Trello và Telegram*

Hành vi theo nhà cung cấp được tóm tắt ở Bảng 3.1. Danh sách "Việc từ cuộc họp" được dịch vụ tự tìm hoặc tạo trên bảng Trello đã cấu hình, mã danh sách được lưu lại cho các lần sau. Thẻ mang tiêu đề công việc và trường hạn nếu ngày đã chốt; mô tả thẻ ghi người thực hiện, hạn hoặc câu hạn gốc, ưu tiên, tên cuộc họp, câu trích nguồn và kết thúc bằng dòng đánh dấu `MeetingMind #<actionItemId>`. Tin Telegram "Công việc mới từ cuộc họp" được gửi qua một bot riêng của MeetingMind và kèm đường dẫn thẻ Trello nếu bước trước thành công — vì vậy Telegram chạy sau Trello. Phương thức `getUpdates` chỉ được dùng để dò cuộc trò chuyện đã nhắn cho bot khi chưa cấu hình mã cuộc trò chuyện.

Bảng 3.1. *Hành vi của mô-đun đồng bộ theo từng nhà cung cấp*

Tiêu chí	Trello	Telegram
Thao tác	Tạo thẻ trong danh sách "Việc từ cuộc họp" (tự tìm hoặc tạo trên bảng cấu hình)	Gửi tin `sendMessage` (HTML) qua bot riêng của MeetingMind
Nội dung	Tiêu đề, người thực hiện, hạn hoặc câu hạn gốc, ưu tiên, cuộc họp, câu trích, dòng đánh dấu	Tiêu đề, người thực hiện, hạn, ưu tiên, cuộc họp, đường dẫn thẻ Trello
Chống trùng	Job theo khoá; tra `external_task_links`; dò thẻ mang dòng đánh dấu trước khi tạo	Job theo khoá; không tra cứu được tin đã gửi
Nơi lưu	`external_task_links` (externalId,	`sync_jobs` (mã tin nhắn)

Tiêu chí	Trello	Telegram
kết quả	externalUrl, syncStatus)	
Thiếu khoá cấu hình	`skipped`, không gọi API	`skipped`, không gọi API

Chống tạo trùng. Tài liệu thiết kế cơ sở dữ liệu đã cảnh báo rằng chống lặp thông điệp nội bộ chưa đủ cho lời gọi ra hệ thống ngoài, nên mô-đun xử lý ở ba lớp. Ở lớp thông điệp, consumer ghi cặp (eventId, tên consumer) vào `processed_messages` trước mọi thao tác khác; chỉ lỗi vi phạm khoá duy nhất mới được coi là "đã xử lý", lỗi cơ sở dữ liệu khác được ném ra để tin được giao lại — bộ chống lặp của Core trước đó nuốt mọi lỗi và có thể làm mất tin, nên được sửa cùng lúc. Ở lớp job, mỗi nhà cung cấp có một bản ghi `sync_jobs` với khoá duy nhất `<actionItemId>:push:<provider>`; job đã thành công thì kết quả cũ được dùng lại, không gọi API lần nữa. Ở lớp nền tảng ngoài, Trello không hỗ trợ khoá idempotency nên lời gọi tạo thẻ có thể đã thành công nhưng mất phản hồi; vì vậy, trước khi tạo, dịch vụ tra `external_task_links` rồi dò trong danh sách đích thẻ mang dòng đánh dấu của công việc và dùng lại nếu tìm thấy:

```
const idempotencyKey = `${payload.idempotencyKey}:${provider}`; //
<id>:push:trello
const job = await this.syncJobService.getOne(null, { idempotencyKey });
if (job?.status === SyncJobStatus.SUCCEEDED) return job.result; // không gọi
API lại
// ...
// Trello không có khoá idempotency: dò dòng đánh dấu trước khi tạo thẻ mới.
const marker = trelloMarker(payload.actionItemId);             //
"MeetingMind #<id>"
let card = await this.trello.findCardByMarker(listId, marker);
if (!card) {
    card = await this.trello.createCard({ listId, name: payload.task.title,
        desc: buildTrelloDescription(payload), due });
}
```

Telegram không có cơ chế tương tự: Bot API không cho tra cứu tin đã gửi, nên nếu `sendMessage` thành công nhưng mất phản hồi, lần thử lại có thể gửi trùng một tin. Nhóm chấp nhận giới hạn này vì một tin báo trùng ít hại hơn một thẻ công việc trùng.

Xử lý lỗi. Mỗi nhà cung cấp chạy độc lập: lỗi API hay cấu hình sai của Trello chỉ làm kết quả Trello là `failed`, tin Telegram vẫn được gửi (không kèm đường dẫn thẻ), và ngược lại. Thiếu khoá cấu hình thì nhà cung cấp đó nhận trạng thái `skipped` chứ không bị coi là lỗi, nên hệ thống vẫn chạy khi chưa kết nối công cụ nào. Với lỗi hạ tầng (cơ sở dữ liệu, RabbitMQ), consumer trả lại bản ghi chống lặp, chờ một nhịp rồi để tin quay lại hàng đợi. Sự kiện `task.sync.completed` được phát thẳng sau khi ghi cơ sở dữ liệu, không qua outbox; điều này chấp nhận được vì Core xử lý lũy đẳng theo `actionItemId` (ghi đè danh sách liên kết), còn nếu phát lỗi thì tin gốc được giao lại và

các job đã thành công chỉ phát lại kết quả cũ. Hệ thống chưa có hàng đợi thư chết cho các lần đồng bộ lỗi.

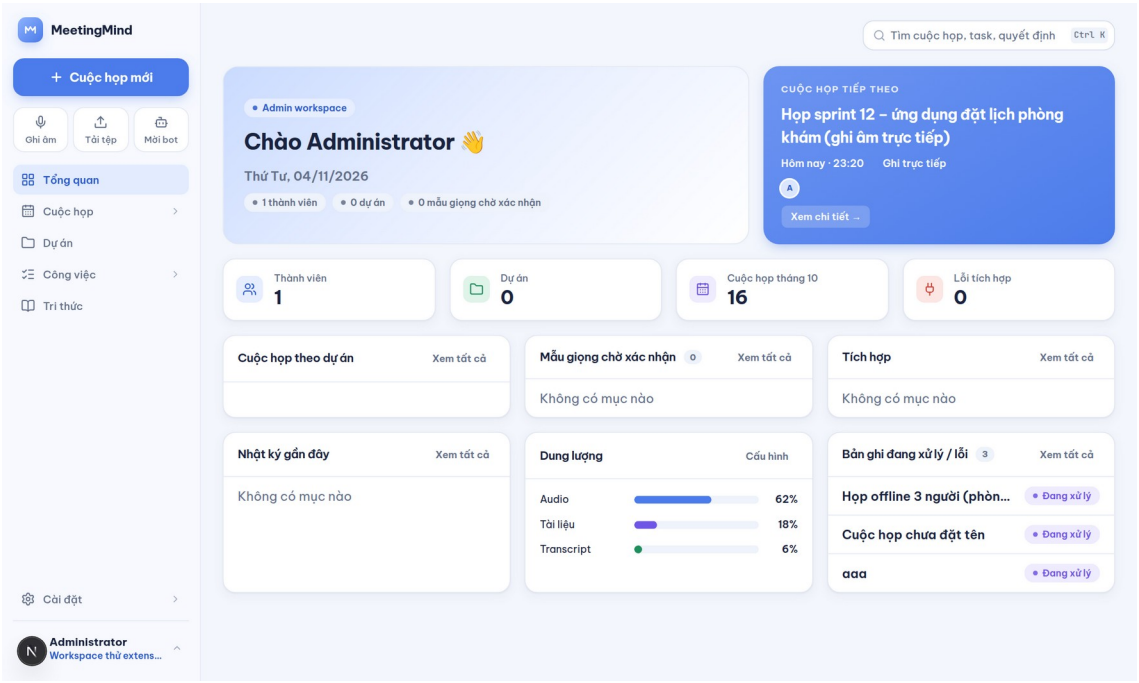
Bảo mật khoá. Khoá API và token Trello, token bot Telegram chỉ được đọc từ biến môi trường (tệp `.env` đã nằm trong `.gitignore`), không ghi vào cơ sở dữ liệu; bảng `integrations` chỉ giữ thông tin không bí mật. Vì khoá Trello nằm trên chuỗi truy vấn còn token Telegram nằm ngay trên đường dẫn URL, mọi thông báo lỗi đều đi qua một hàm xoá bí mật trước khi ghi log, ghi cơ sở dữ liệu hay gửi về Core; nhóm đã kiểm tra log và thông báo lỗi không chứa khoá.

Lỗi khung mẫu. Khi bật RabbitMQ cho integration-service, nhóm gặp một lỗi kế thừa từ khung mẫu: các interceptor và bộ lọc ngoại lệ HTTP được gắn toàn cục nên cũng chạy với handler RabbitMQ, vốn không có đối tượng yêu cầu và phản hồi HTTP; handler bị sập và tin bị trả lại hàng đợi khoảng 1.500 lần mỗi giây. Các thành phần này được sửa để bỏ qua ngữ cảnh không phải HTTP.

Kết quả kiểm thử. Mô-đun có 20 ca kiểm thử đơn vị Jest, tổng hợp ở mục Kiểm thử phần mềm. Trong kiểm thử đầu-cuối trên Docker Compose với tài khoản Trello và bot Telegram thật, duyệt công việc "Viết tài liệu API bằng Swagger" thì sau 3,4 giây thẻ được tạo trên bảng "MeetingMind", trong danh sách "Việc từ cuộc họp" do hệ thống tự tạo; tin Telegram kèm đường dẫn thẻ được gửi; Core lưu cả hai kết quả và giao diện hiển thị như Hình 3.2. Một lần duyệt trước đó, khi chưa cấu hình khoá Trello, cho kết quả Trello `skipped` và tin Telegram đến sau khoảng 2 giây, đúng thiết kế bỏ qua nhà cung cấp thiếu cấu hình.

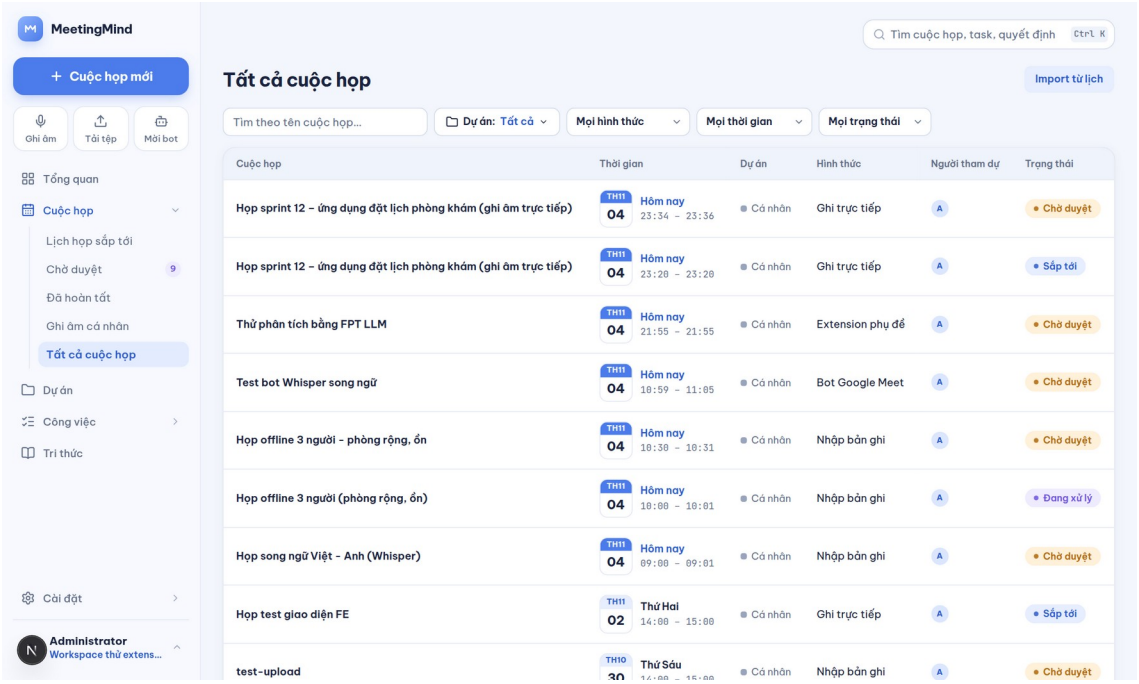
3.4. Giao diện web

Giao diện xây dựng bằng Next.js 16 (App Router), React 19, Tailwind CSS 4 và shadcn/Radix UI. Trình duyệt chỉ gọi `/api/...` cùng nguồn gốc, Next.js chuyển tiếp thẳng tới core-service (cổng 3010) nên không phải cấu hình CORS; cũng như extension, giao diện hiện chưa đi qua api-gateway. Giao diện được thiết kế trước dưới dạng bản mẫu, sau đó mới nối backend qua một lớp chuyển đổi: `live.ts` gom mọi lời gọi API, `adapters.ts` chuyển dữ liệu backend sang kiểu giao diện, nên các thành phần không phải viết lại. Có phiên đăng nhập thì chạy chế độ *live*; chưa đăng nhập, nút "Xem demo" mở chế độ *demo* với dữ liệu mẫu. Ở chế độ live, sau mỗi thao tác giao diện nạp lại từ backend thay vì tự sửa trạng thái, vì backend quyết định kết quả. Các trang mà backend chưa có API — ghi âm trực tiếp trên web, hỏi đáp tri thức (RAG), khách mời, mẫu giọng, tích hợp, mẫu biên bản, lưu trữ, nhật ký — mới là giao diện mẫu, chỉ có dữ liệu ở chế độ demo.



Hình 3.3. *Màn hình tổng quan của người quản lý dự án*

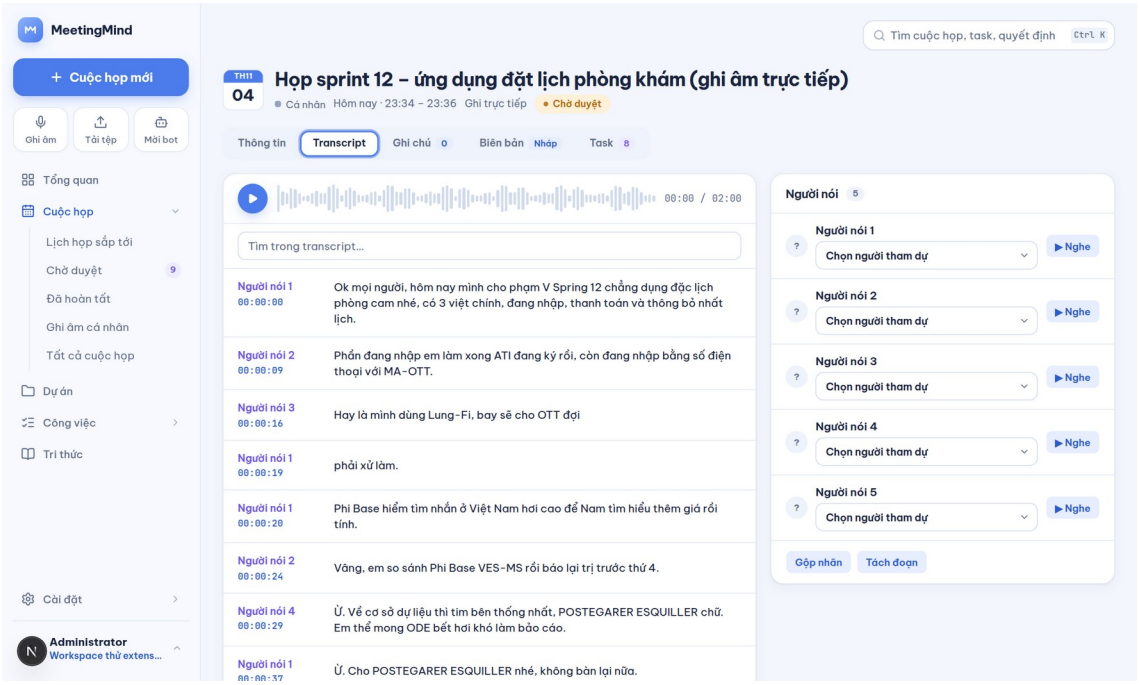
Màn hình tổng quan (Hình 3.3) thay đổi theo vai trò. Với PM, trang hiển thị chỉ số nhanh (cuộc họp trong tuần, mục chờ duyệt, task đang mở, task quá hạn), lịch họp sắp tới, danh sách chờ duyệt và việc của tôi. Ô cảnh báo được tính ở giao diện theo quy tắc đơn giản (quá hạn, bị chặn, chưa có người thực hiện); phát hiện rủi ro tự động ở backend chưa triển khai.



Hình 3.4. *Danh sách cuộc họp với bộ lọc và giai đoạn xử lý*

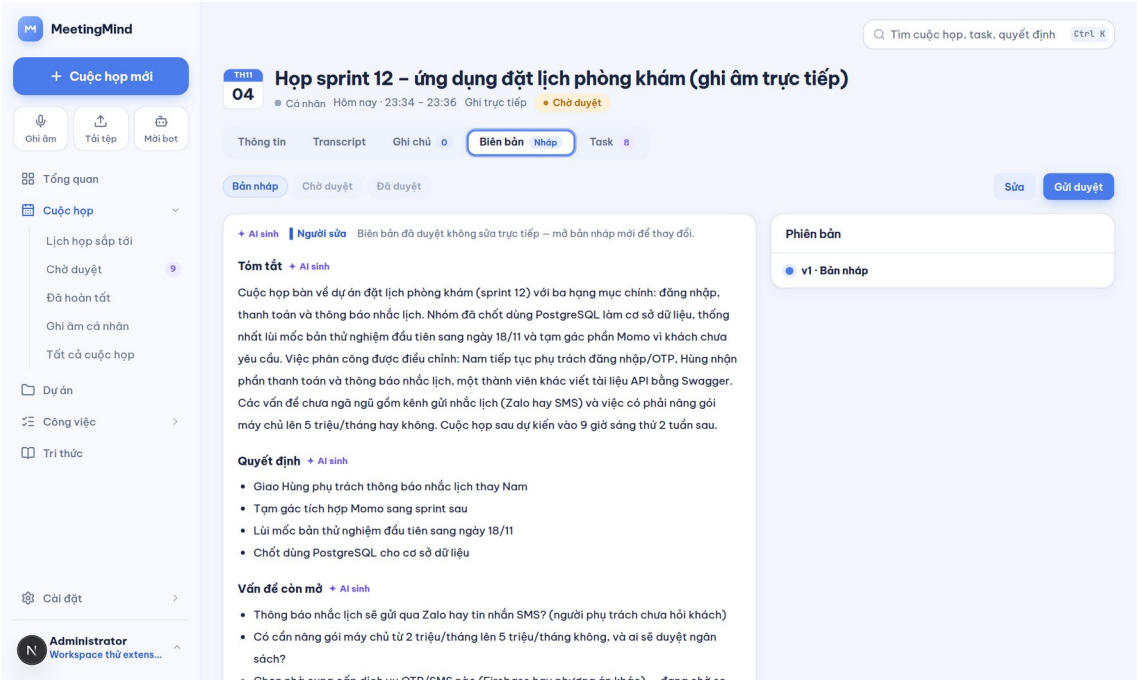
Danh sách cuộc họp (Hình 3.4) cho phép tìm kiếm, lọc theo dự án, hình thức (tải tệp, bot, extension, họp trực tiếp), thời gian, trạng thái. Mỗi cuộc họp mang giai đoạn

hiển thị do backend gộp từ trạng thái cuộc họp, job xử lý gần nhất và trạng thái biên bản: đã lên lịch, đang xử lý, lỗi, chờ duyệt, hoàn tất.



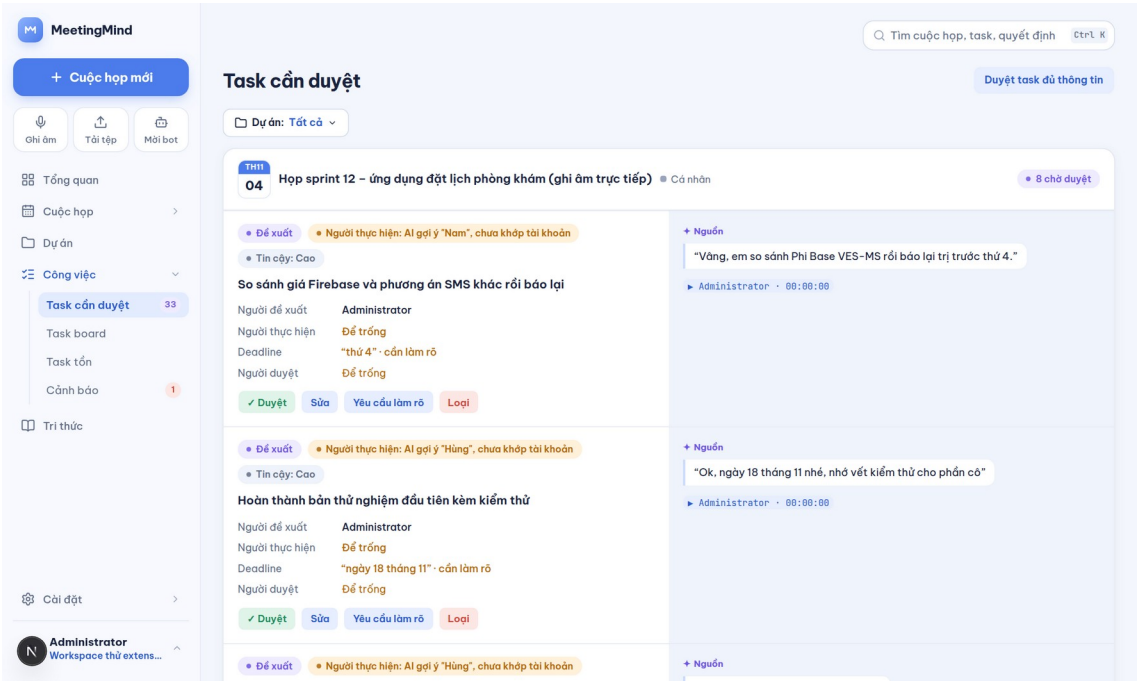
Hình 3.5. *Màn hình transcript với nhãn ngôn ngữ, cảnh báo nói chồng và gán người nói*

Màn hình transcript (Hình 3.5) hiển thị từng đoạn với người nói, mốc thời gian và ô tìm kiếm. Khi cuộc họp có từ hai thứ tiếng, mỗi đoạn có nhãn mã ngôn ngữ (VI, EN); đoạn có cờ nói chồng mang nhãn "Nói chồng · nên nghe lại". Cột bên phải liệt kê người nói, cho phép gán nhãn tự động với thành viên. Thanh phát âm thanh hiện mới mô phỏng; API cấp URL ký đã có nhưng giao diện chưa nối vào.



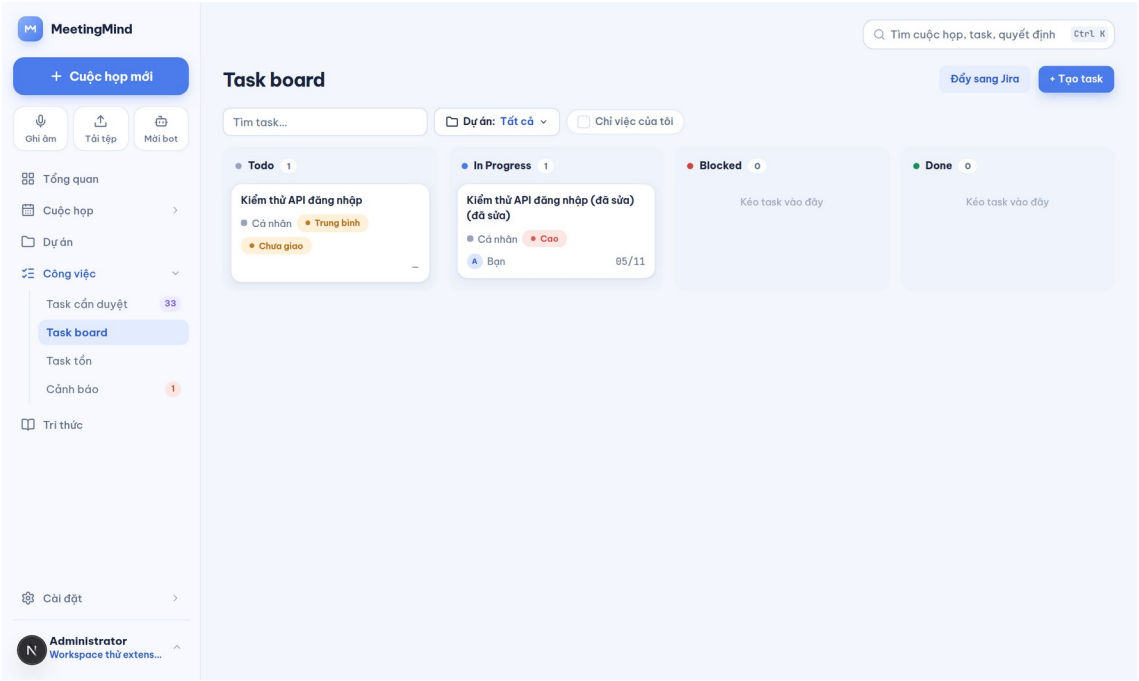
Hình 3.6. *Màn hình biên bản và vòng duyệt theo phiên bản*

Màn hình biên bản (Hình 3.6) hiển thị tóm tắt, quyết định, vấn đề còn mở, công việc và phiên bản; nút thao tác đổi theo trạng thái: "Gửi duyệt" với bản nháp, "Duyệt biên bản" và "Từ chối" với bản chờ duyệt, "Mở bản nháp mới" với bản đã duyệt.



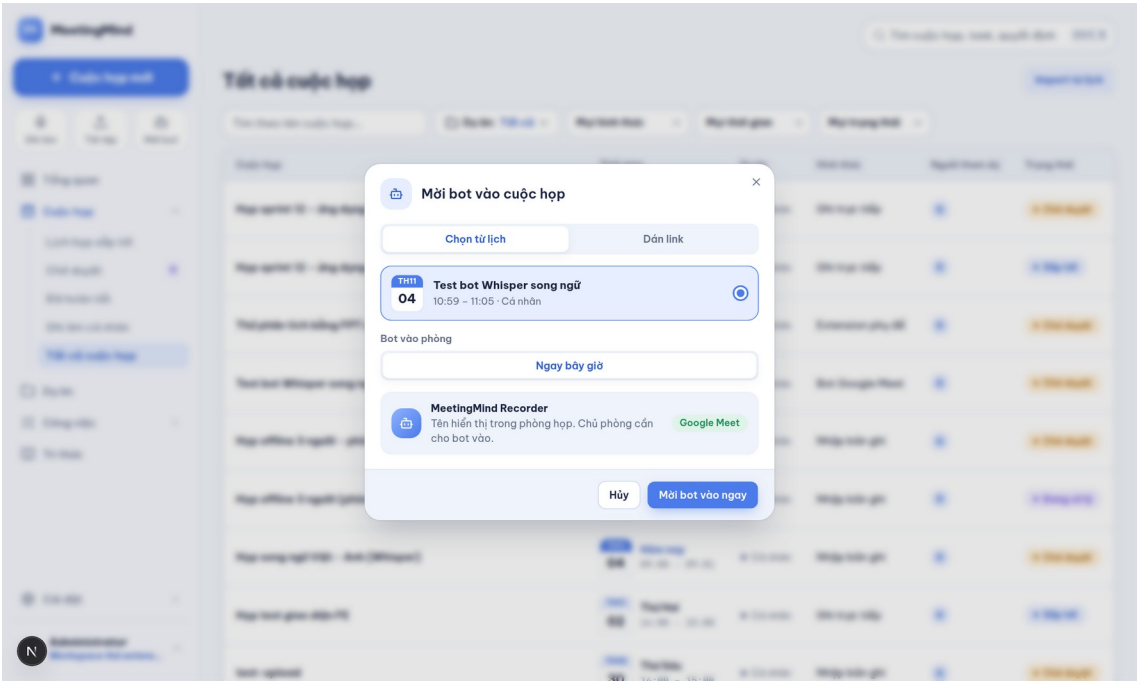
Hình 3.7. *Hàng đợi duyệt đề xuất công việc do AI trích xuất*

Hàng đợi duyệt (Hình 3.7) nhóm đề xuất theo cuộc họp. Mỗi thẻ có dấu hiệu nội dung do AI sinh, mức tin cậy, các trường sửa được (người thực hiện, hạn) và câu trích nguyên văn làm bằng chứng; người duyệt chọn "Duyệt", "Sửa" hoặc "Loại". Đây là điểm kiểm soát của con người: chỉ sau khi duyệt, đề xuất mới thành task chính thức.



Hình 3.8. *Bảng công việc theo trạng thái*

Bảng công việc (Hình 3.8) trình bày task theo bốn cột Todo, In Progress, Blocked, Done kèm dự án, hạn, ưu tiên; mở một task sẽ hiện ngăn chi tiết với bình luận.



Hình 3.9. *Hộp thoại mời bot ghi âm vào cuộc họp Google Meet*

Hộp thoại mời bot (Hình 3.9) cho chọn cuộc họp trực tuyến trong lịch hoặc dán đường dẫn Google Meet mới; ở chế độ live chỉ có lựa chọn "Ngay bây giờ" vì Core chưa có bộ hẹn giờ cho bot. Sau khi mời, trang cuộc họp hỏi trạng thái bot mỗi 4 giây, hiển thị "Chờ chủ phòng cho vào", "Bot đang ghi âm" hoặc lý do thất bại cùng nút "Dừng bot".

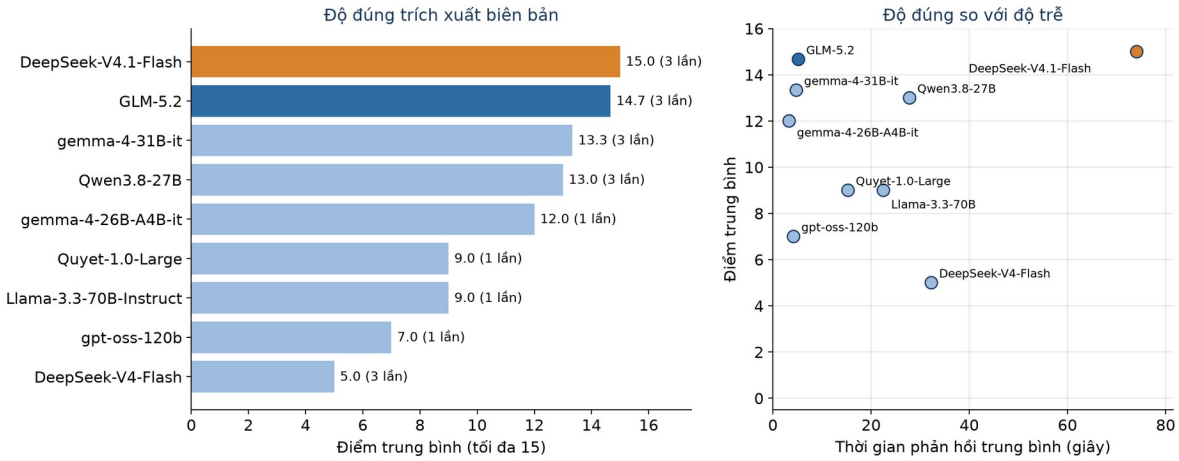
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Lựa chọn mô hình ngôn ngữ lớn

Nhóm thử tất cả các LLM mà khoá API được cấp quyền trên FPT AI Factory. Tám mô hình thuộc nhóm "bán lại" (Azure gpt-5.6, Google Gemini, Alibaba Qwen/DeepSeek Pro, Z.ai GLM-5.3, Minimax) trả lỗi 402 do tài khoản chưa nạp tiền cho nhóm này; các mô hình này đồng thời gửi dữ liệu ra ngoài hạ tầng FPT nên không được ưu tiên. Chín mô hình do FPT tự vận hành được đánh giá trên transcript chuẩn của kịch bản, cùng một prompt hệ thống, nhiệt độ 0,2. Năm mô hình tốt nhất ở lượt đầu được chạy lại ba lần để đánh giá độ ổn định. Kết quả ở Bảng 4.1 và Hình 4.1.

Bảng 4.1. *So sánh các LLM trên bài trích xuất biên bản (transcript chuẩn, tối đa 15 điểm)*

Mô hình	Số lần chạy	Điểm từng lần	Trung bình	Thời gian TB (s)	Chi phí/lần (USD)
DeepSeek-V4.1-Flash	3	15, 15, 15	15,0	74,1	0,0052
GLM-5.2	3	15, 14, 15	14,7	5,2	0,0063
gemma-4-31B-it	3	14, 13, 13	13,3	4,8	0,0006
Qwen3.8-27B	3	13, 12, 14	13,0	27,8	0,0036
gemma-4-26B-A4B-it	1	12	12,0	3,3	0,0006
Quyet-1.0-Large	1	9	9,0	15,3	0,0001
Llama-3.3-70B-Instruct	1	9	9,0	22,5	0,0006
gpt-oss-120b	1	7	7,0	4,2	0,0007
DeepSeek-V4-Flash	3	0, 0, 15	5,0	32,3	0,0024



Hình 4.1. *Độ đúng trích xuất và độ trễ của các LLM trên FPT AI Factory*

DeepSeek-V4.1-Flash là mô hình duy nhất đạt trọn 15/15 điểm ở cả ba lần chạy, không dính bẫy nào: nhận ra đề xuất Firebase chưa phải quyết định, lấy đúng hạn

18/11 sau đính chính, giao việc nhắc lịch cho Hùng thay vì Nam, bỏ qua chuyện bóng đá và suy ra "Người nói 4" là Phương. Đổi lại, đây là mô hình suy luận nên mất 45–90 giây mỗi lần. GLM-5.2 đạt gần tương đương (14,7) với thời gian chỉ khoảng 5 giây; lỗi duy nhất là một lần bỏ sót việc bị đổi người thực hiện. Các mô hình nhỏ hơn (gemma, Qwen 27B) thường bỏ sót các quyết định dạng phủ định ("chưa làm MoMo") hoặc công việc gắn với người chủ trì. Các mô hình đa năng thế hệ trước (Llama 3.3 70B, gpt-oss-120b) và mô hình tiếng Việt Quyet-1.0-Large bỏ sót nhiều công việc.

Thực nghiệm cũng phát hiện hai vấn đề kỹ thuật quan trọng cho việc tích hợp. Thứ nhất, khi bật chế độ ép đầu ra JSON (response_format), DeepSeek-V4.1-Flash trả về nội dung rỗng (toàn bộ đầu ra nằm ở phần suy luận) và DeepSeek-V4-Flash trả về cấu trúc gọi hàm lặp lại transcript; vì vậy hệ thống không dùng chế độ này mà tự bóc và kiểm tra JSON. Thứ hai, DeepSeek-V4-Flash không ổn định: hai trong ba lần chạy rơi vào vòng lặp sinh văn bản đến khi hết giới hạn token.

Từ kết quả trên, hệ thống chọn **DeepSeek-V4.1-Flash làm mô hình chính** vì độ chính xác và độ ổn định cao nhất, chi phí khoảng 0,005 USD (khoảng 130 đồng) mỗi cuộc họp; độ trễ một phút không ảnh hưởng vì phân tích chạy nền sau bước phiên âm. **GLM-5.2 làm mô hình dự phòng** khi mô hình chính lỗi, quá thời gian hoặc trả JSON hỏng. Cả hai đều do FPT tự vận hành, dữ liệu cuộc họp không rời hạ tầng trong nước.

4.2. Đánh giá đầu-cuối từ âm thanh đến công việc

Ở thực nghiệm này, transcript do Whisper nhận dạng và mô-đun tách người nói gán nhãn được đưa thẳng vào bộ phân tích với mô hình đã chọn, đúng như luồng chạy thật. Mỗi transcript được phân tích hai lần: với nhãn tự động ("Người nói 1…4") và với tên thật được gán theo cách ghép người nói tốt nhất, mô phỏng thao tác "gán tên người nói" của người dùng trên giao diện. Bảng 4.2 trình bày điểm nghiêm ngặt và điểm sau chuẩn hoá cách viết thứ trong tuần.

Bảng 4.2. *Điểm trích xuất khi đầu vào là transcript do máy nhận dạng (tối đa 15)*

Điều kiện	Cấu hình STT	Nhãn tự động	Đã gán tên	Mô hình phân tích
Transcript chuẩn (đối chứng)	—	—	15 / 15	DeepSeek-V4.1-Flash
Sạch	small	10 / 11	12 / 13	DeepSeek-V4.1-Flash
Sạch	small + từ điển	11 / 12	9 / 10	DeepSeek-V4.1-Flash
Sạch	medium	11 / 11	15 / 15	DeepSeek-V4.1-Flash
Nhiễu trắng 20 dB	small	10 / 11	10 / 11	DeepSeek-V4.1-Flash

Điều kiện	Cấu hình STT	Nhãn tự động	Đã gán tên	Mô hình phân tích
Nhiễu trắng 10 dB	small	8 / 9	12 / 13	DeepSeek-V4.1-Flash
Nhiễu trắng 10 dB	small + từ điển	6 / 6	13 / 13	DeepSeek-V4.1-Flash
Nhiễu trắng 10 dB	medium	12 / 12	14 / 14	DeepSeek-V4.1-Flash
Nhiễu trắng 5 dB	small	5 / 5	8 / 8	GLM-5.2 (dự phòng)
Đông người 10 dB	small	8 / 9	11 / 12	DeepSeek-V4.1-Flash
Đông người 10 dB	small + từ điển	8 / 8	13 / 14	DeepSeek-V4.1-Flash
Đông người 10 dB	medium	12 / 12	13 / 13	DeepSeek-V4.1-Flash
Nói chồng + đoạn nhiễu	small	6 / 7	13 / 14	DeepSeek-V4.1-Flash

Ghi chú: mỗi ô dạng "nghiêm ngặt / sau chuẩn hoá cách viết thứ"; mỗi điều kiện chạy một lần.

Có ba nhận xét chính. Thứ nhất, so với 15/15 trên transcript chuẩn, điểm trên transcript máy giảm còn khoảng 8–13 điểm với Whisper small: lỗi nhận dạng lan sang bước trích xuất, đặc biệt ở các công việc gắn với thuật ngữ bị nghe sai (hạn "cuối tuần này" bị nghe thành "cuối tôm nay" nên công việc viết tài liệu mất thời hạn đúng). Thứ hai, **gán tên người nói giúp tăng điểm rõ rệt** (trung bình khoảng 3 điểm ở các điều kiện có nhiễu): khi chỉ có nhãn "Người nói 1…4", mô hình khó xác định ai nhận việc nếu câu giao việc không nêu tên. Điều này khẳng định giá trị của bước gán tên người nói; từ kết quả này, hệ thống được bổ sung thao tác "phân tích lại" để chạy lại AI sau khi người dùng gán tên. Thứ ba, chất lượng nhận dạng tốt hơn kéo theo trích xuất tốt hơn: với Whisper medium ở điều kiện sạch, hệ thống đạt lại 15/15 điểm khi đã gán tên.

Một quan sát đáng chú ý là LLM tự sửa được nhiều lỗi nhận dạng nhờ ngữ cảnh: transcript ghi "phai bây sơ", "EAS-MS", "so gơ", nhưng công việc được trích xuất lại ghi đúng "Firebase", "ESMS", "Swagger". Ngược lại, LLM không thể khôi phục thông tin đã mất hẳn (thời hạn bị nghe sai). Ở điều kiện nhiễu 5 dB, mô hình chính suy luận quá lâu nên cổng dịch vụ trả lỗi hết thời gian chờ (HTTP 524) và hệ thống tự chuyển sang GLM-5.2, chứng tỏ cơ chế dự phòng hoạt động đúng; đồng thời cho thấy cần chuyển lời gọi LLM sang chế độ truyền luồng. Sau thực nghiệm, lời gọi đã được chuyển sang chế độ luồng và kiểm chứng lại với cả hai mô hình trên API thật.

Các ô của Bảng 4.2 chỉ chạy một lần nên chịu ảnh hưởng của tính ngẫu nhiên của LLM (ví dụ ô "small + từ điển, sạch, đã gán tên" chỉ đạt 9 điểm dù nhận dạng thuật ngữ tốt hơn); cần nhiều lần chạy hơn để so sánh các cấu hình STT với độ tin cậy thống kê.

CHƯƠNG 5. KẾT QUẢ NỘI DUNG VÀ ĐÁNH GIÁ

5.1. Đối chiếu với kết quả cần đạt

Bảng 5.1. *Đối chiếu kết quả Nội dung 4 với đề cương*

Kết quả cần đạt theo đề cương	Kết quả thực hiện	Đánh giá
Agent tự động tạo biên bản họp	Bộ phân tích LLM (DeepSeek-V4.1-Flash, dự phòng GLM-5.2) tạo tóm tắt, quyết định, vấn đề còn mở, công việc kèm người thực hiện, thời hạn, câu trích dẫn; Core tạo biên bản nháp theo phiên bản và hàng đợi duyệt	Đạt
Trích xuất action items	15/15 điểm trên transcript chuẩn ở cả 3 lần chạy; 8–15 điểm trên transcript do máy nhận dạng tuỳ điều kiện âm thanh	Đạt
Đồng bộ thành công công việc lên Jira/Trello	integration-service tạo thẻ Trello cho công việc đã duyệt (người thực hiện, hạn, ưu tiên, câu trích nguồn), chống tạo trùng ba lớp, trả kết quả về Core và giao diện; với tài khoản Trello thật, thẻ được tạo sau 3,4 s kể từ khi duyệt. Jira chưa hiện thực	Đạt (qua Trello; Jira chưa làm)
Cảnh báo công việc mới qua Telegram	Bot Telegram riêng của MeetingMind gửi tin "Công việc mới từ cuộc họp" kèm đường dẫn thẻ Trello; đã chạy với bot thật	Đạt
(Bổ sung) Con người duyệt trước khi giao việc	Duyệt, sửa, từ chối đề xuất trên giao diện; chỉ mục đã duyệt mới thành công việc, quyết định chính thức	Đạt

5.2. Tồn tại và hướng hoàn thiện

Nội dung 4 đạt các kết quả cần đạt của đề cương. Phần AI Agent đạt và vượt yêu cầu về độ tin cậy nhờ cơ chế duyệt và đối chiếu bằng chứng. Phần đồng bộ đáp ứng yêu cầu "Jira/Trello" bằng Trello, kèm cảnh báo công việc mới qua Telegram, và đã được kiểm chứng với tài khoản thật; thiết kế chống tạo trùng được áp dụng đúng như dự kiến, kể cả tình huống lời gọi tạo thẻ thành công nhưng mất phản hồi.

Tuy vậy, phần đồng bộ mới ở mức sản phẩm khả dụng tối thiểu. Mỗi nền tảng chỉ có một kết nối cấu hình qua biến môi trường, chưa có màn hình kết nối OAuth và chọn bảng theo từng workspace; đồng bộ một chiều, chưa cập nhật trạng thái hai chiều; chưa có hàng đợi thư chết cho các lần đồng bộ lỗi; tin Telegram có thể bị gửi trùng nếu mất phản hồi vì Bot API không cho tra cứu tin đã gửi. Jira chưa được hiện

thực do nhóm chỉ có hai thành viên và ưu tiên hoàn thiện nền tảng xử lý cùng chất lượng mô hình AI trước. Vì integration-service đã có sẵn cơ chế job theo khoá chống lặp, bảng liên kết ngoài và sự kiện kết quả, bổ sung Jira chủ yếu là viết thêm một client cho REST API của Jira. Hướng hoàn thiện tiếp theo là màn hình kết nối theo workspace, đồng bộ hai chiều qua webhook và tích hợp Jira.

TÀI LIỆU THAM KHẢO

[1] Atlassian (2026), *Jira Cloud platform REST API v3*, https://developer.atlassian.com/cloud/jira/platform/rest/v3/, truy cập ngày 04/11/2026.

[2] Atlassian (2026), *Trello REST API*, https://developer.atlassian.com/cloud/trello/rest/, truy cập ngày 04/11/2026.

[3] Brown T. B., Mann B., Ryder N., Subbiah M., Kaplan J., et al. (2020), "Language models are few-shot learners", *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 1877–1901.

[4] DeepSeek-AI (2024), *DeepSeek-V3 Technical Report*, arXiv:2412.19437.

[5] FPT Smart Cloud (2026), *FPT AI Factory – Serverless Inference documentation*, https://docs.fptcloud.com/docs/category/fpt-ai-factory/, truy cập ngày 04/11/2026.

[6] Hardt D. (Ed.) (2012), *The OAuth 2.0 Authorization Framework*, RFC 6749, Internet Engineering Task Force (IETF).

[7] Ji Z., Lee N., Frieske R., Yu T., Su D., Xu Y., Ishii E., Bang Y. J., Madotto A., Fung P. (2023), "Survey of hallucination in natural language generation", *ACM Computing Surveys*, 55(12), pp. 1–38.

[8] Team GLM (2024), *ChatGLM: A family of large language models from GLM-130B to GLM-4 All Tools*, arXiv:2406.12793.

[9] Telegram (2026), *Telegram Bot API*, https://core.telegram.org/bots/api, truy cập ngày 04/11/2026.